

Overview

Agile is a software development methodology that focuses on developing software in small, manageable parts rather than completing the entire project at once. It emphasizes continuous customer feedback, teamwork, flexibility, and frequent delivery of working software.

Instead of following one long development cycle, Agile divides the project into short iterations called Sprints, allowing teams to deliver features quickly and improve them based on customer feedback.

Definition

Agile is an iterative and incremental software development approach that helps teams deliver high-quality software quickly while adapting to changing customer requirements.

Example

Suppose you are developing an Online Shopping Application.

- Traditional Method: Build the complete application first, then deliver it after several months.
- Agile Method: Deliver the Login page in Sprint 1, Product Catalog in Sprint 2, Shopping Cart in Sprint 3, and Payment Module in Sprint 4. Customers provide feedback after each sprint, and improvements are made continuously.

2. Why Agile?

Traditional software development often faced several problems:

- Long development cycles
- Late delivery of software
- Difficult to accommodate changing requirements
- High development cost
- Customer involvement only at the end

Agile solves these problems by:

- Delivering software quickly
- Encouraging customer participation
- Accepting changing requirements
- Improving product quality
- Reducing project risks

3. Agile Manifesto

The Agile Manifesto was created in 2001 by 17 software developers. It defines four core values of Agile software development.

Four Agile Manifesto Values

1. Individuals and Interactions over Processes and Tools

Meaning: People and communication are more important than strict processes or tools.

Example: Team members discussing solutions together is more effective than relying only on documents.

2. Working Software over Comprehensive Documentation

Meaning: Delivering functional software is more valuable than creating excessive documentation.

Example: A working login page provides more value than a lengthy design document.

3. Customer Collaboration over Contract Negotiation

Meaning: Customers should actively participate throughout development rather than only during contract discussions.

Example: Customers review each sprint and suggest improvements.

4. Responding to Change over Following a Plan

Meaning: Agile welcomes changing requirements even during development.

Example: If a customer requests a new payment option during development, the team can add it in the next sprint.

4. Agile Principles

The Agile Manifesto is supported by **12 principles**. Some important principles are:

- Deliver working software frequently.
- Welcome changing requirements.
- Collaborate closely with customers.
- Build projects around motivated individuals.
- Encourage face-to-face communication.
- Measure progress through working software.
- Maintain a sustainable development pace.
- Continuously improve processes.
- Focus on technical excellence.
- Keep designs simple.
- Self-organizing teams produce the best solutions.
- Regularly reflect and improve team performance.

5. Agile Life Cycle

The Agile development process typically follows these stages:

1. Requirements Gathering
2. Planning
3. Design
4. Development
5. Testing

6. Deployment

7. Review

8. Maintenance

Since Agile is iterative, these steps repeat for every sprint.

6. Scrum Framework

Scrum is the most popular Agile framework used for managing software development projects.

It divides work into **short iterations called Sprints**, usually lasting **2 to 4 weeks**.

Scrum consists of:

- Roles
- Events
- Artifacts

7. Scrum Roles

A. Product Owner

The **Product Owner** represents the customer and decides what features should be developed.

Responsibilities

- Defines product vision
- Maintains Product Backlog
- Prioritizes requirements
- Communicates with stakeholders
- Maximizes product value

B. Scrum Master

The **Scrum Master** ensures the Scrum process is followed correctly.

Responsibilities

- Facilitates Scrum meetings
- Removes obstacles
- Coaches the team
- Helps implement Agile practices
- Protects the team from distractions

C. Development Team

The **Development Team** designs, develops, tests, and delivers the software.

Responsibilities

- Analyze requirements
- Develop features
- Test the software
- Fix defects
- Deliver a working product increment

8. Scrum Events

Sprint

A Sprint is a fixed period (usually **2–4 weeks**) during which selected work is completed.

At the end of every sprint, a working product increment is delivered.

Sprint Planning

During Sprint Planning, the team decides:

- What work will be completed

- How the work will be completed
- Sprint goals

Daily Scrum

A **15-minute daily meeting** where each team member answers:

1. What did I complete yesterday?
2. What will I do today?
3. Are there any obstacles?

Purpose:

- Track progress
- Identify issues
- Coordinate work

Sprint Review

Held at the end of the sprint.

Purpose:

- Demonstrate completed work
- Collect customer feedback
- Update the Product Backlog if needed

Sprint Retrospective

Conducted after the Sprint Review.

Purpose:

- Discuss what went well
- Identify problems
- Plan improvements for the next sprint

9. Scrum Artifacts

Product Backlog

A prioritized list of all project requirements maintained by the Product Owner.

Example:

- User Registration
- Login
- Product Search
- Shopping Cart
- Online Payment

Sprint Backlog

A subset of Product Backlog items selected for the current sprint.

Example:

- Login
- Forgot Password
- User Dashboard

Product Increment

The completed and tested functionality delivered at the end of a sprint.

Each increment should be usable and potentially releasable.

10. User Story

A **User Story** is a short description of a software feature from the user's perspective.

Format

As a <user>
I want <feature>
So that <benefit>

Example

As a customer,
I want to log in using my email and password,
So that I can securely access my account.

Another example:

As a student,
I want to view my exam results online,
So that I can check my marks easily.

11. Sprint

A **Sprint** is a short, fixed development cycle.

Duration

Usually **2–4 weeks**.

Activities During a Sprint

- Planning
- Designing
- Coding
- Testing
- Review
- Delivery

Outcome: A working software increment.

12. Burndown Chart

A **Burndown Chart** is used to measure the remaining work during a sprint.

It helps the team determine whether they are on schedule to complete the sprint.

Purpose

- Monitor daily progress
- Track remaining tasks
- Predict sprint completion
- Identify delays early

13. Advantages (Benefits) of Agile

- Faster software delivery
- Better customer satisfaction
- Continuous customer feedback
- Improved teamwork
- Better product quality
- Easy to accommodate changing requirements
- Reduced project risks
- Faster issue identification
- Continuous improvement
- Increased transparency

14. Agile vs Traditional (Waterfall)

Agile	Waterfall
Iterative development	Sequential development
Flexible to changes	Difficult to change requirements
Frequent software delivery	Delivery at the end of the project
Continuous customer involvement	Limited customer involvement
Short development cycles	Long development cycles

Agile

Lower project risk

Waterfall

Higher project risk

15. Real-Time Example of Agile

Project: Online Food Delivery Application

Sprint 1

- User Registration
- Login

Sprint 2

- Restaurant Listing
- Search Food

Sprint 3

- Cart
- Order Placement

Sprint 4

- Online Payment
- Order Tracking

After each sprint, customers test the application and provide feedback. The development team uses this feedback to improve the next sprint.